

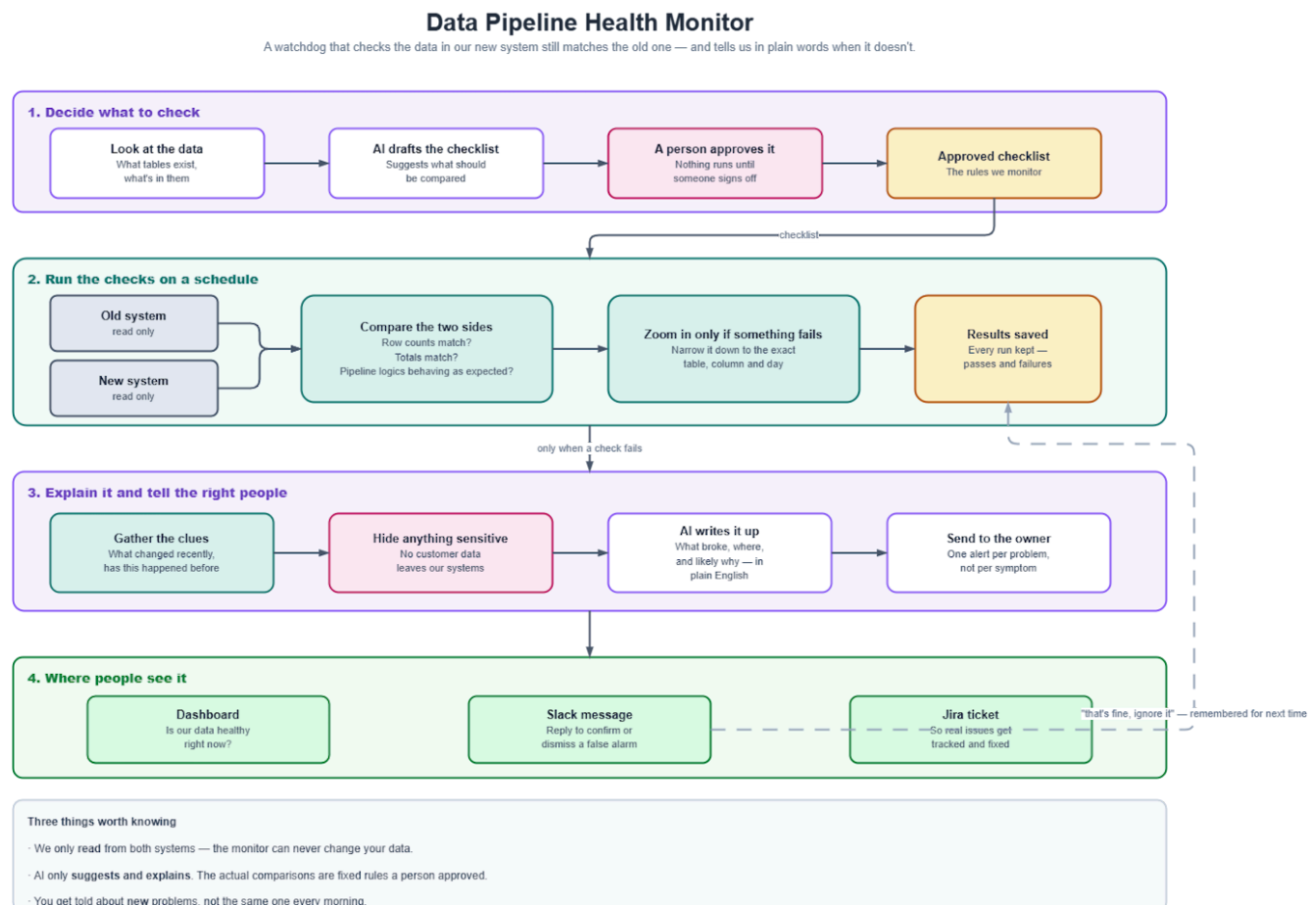
Data Pipeline Health Monitor

1. The problem

Right now, pipeline checks are written by hand, after something has already gone wrong. Worse, we can't say what isn't being checked — so no alerts doesn't mean everything is healthy. The clearest example: a dimension that's supposed to keep history quietly starts overwriting it instead. The history disappears, but the target still matches the source, so every check stays green.

The real question we need answered is simple: is this pipeline's logic doing what it's supposed to? That's a single-source question, and we can answer it with plain SQL against one warehouse.

2. Architecture



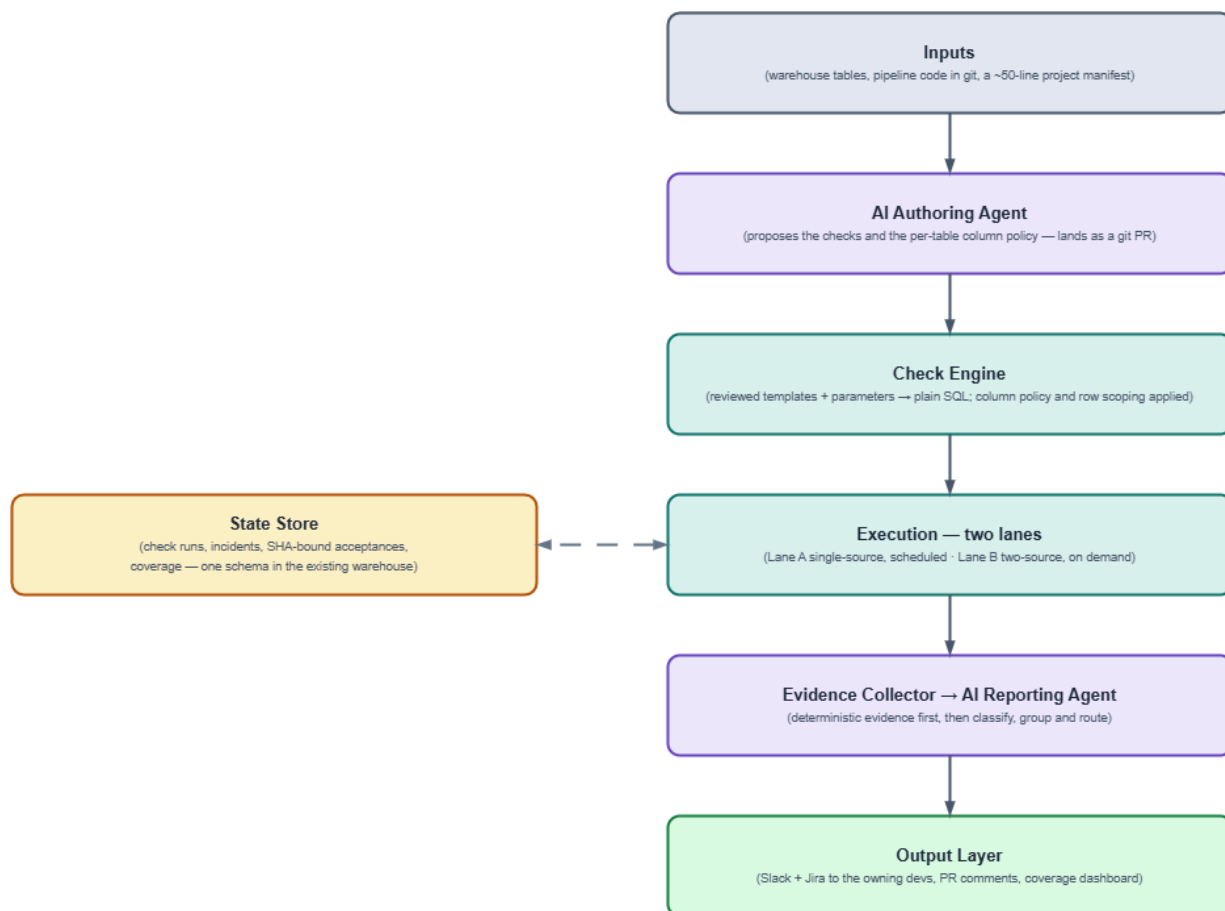
2.1 Two lanes

We split checks into two lanes:

	Lane A — single source	Lane B — two source
Question	Is the pipeline logic correct?	Do these two sources agree?
Data	One dataset, or layers within one warehouse	Source A against source B
Cost	Cheap SQL	The expensive one
Trigger	Scheduled — cron or a post-run hook	On demand — CLI or a pull request
Reporting	Full: Slack + Jira to owning devs, deduplicated	Returned to whoever ran it

Lane A runs unattended, so it needs extra machinery — deduplication, sign-offs, ownership — a failing check should create one incident with updates, not a new ticket every run. Lane B is expensive, but a person is already watching the output, so none of that machinery is needed there. Every check just declares its own lane and trigger. One engine, two speeds.

2.2 Components



- Check engine — turns reviewed templates and parameters into plain SQL. That SQL is the check.
- AI authoring agent — reads schemas, pipeline code, and lineage, then proposes checks and column rules so we're not hand-writing hundreds of them. Output lands as a git PR.
- Evidence collector — runs before any AI call. Pulls schema diffs, load history, git history on both sides, past incidents, and the audit columns that show which run caused a problem.

- AI reporting agent — sorts failures into known categories, groups related ones into a single incident, and routes it to the right team with a likely author.
- State store — one schema in our existing warehouse. No new database.

2.3 The core idea

A check is defined by: lane, trigger, what it covers, the relationship it's asserting, scope, column rules, and tolerance.

The "relationship" part is what makes this flexible. A check isn't just "A equals B" — it asserts something specific: one row per key, valid SCD2 history, conservation (input equals output plus named losses), every foreign key resolves, values fall in range, a business rule holds, or an aggregate matches. Those are all Lane A. Lane B checks for an exact match, a filtered subset, or a superset. There's also "unvalidated," meaning we have no way to check it yet — that still gets reported, because knowing what we can't check matters too.

2.4 Why plain SQL instead of hashing

Since both sides sit in the same engine, one SQL statement can show exactly which rows differ and which column caused it:

```
select coalesce(a.order_id, b.order_id) as order_id, a.amount as a_amount, b.amount as b_amount
from staging.orders a full outer join prod.orders b using (order_id)
where a.order_id is null or b.order_id is null or a.amount is distinct from b.amount -- etl_updated_at,
_loaded_at, batch_id deliberately excluded
```

That means we don't need hash comparisons, a separate step to pinpoint failures, or any cross-database logic. Hashing survives in only one place: making sure closed SCD versions never change, since there's no second table to compare against there.

Column rules matter just as much. Audit and ETL columns change on every run, so if we don't exclude them, every row looks different. We split columns into four types — key, business, audit/ETL (excluded by default), and derived — and every run reports exactly which columns were compared and which were skipped. We also scope which rows we check, so a scheduled check never reads data mid-load.

2.5 The SCD suite

This is eleven checks per dimension, built from a business key plus the type-1/type-2/validity columns: exactly one current row per key, no overlapping or missing date ranges, a new version only appears when something tracked actually changed — and just as important, a tracked change did create a new version. That last one is what catches a dimension quietly turning into an overwrite. On top of that: surrogate key uniqueness, foreign keys resolve, and closed versions never change.

This only works once audit columns are excluded — otherwise every run looks like a change, and the check tells us nothing.

2.6 Why the checks can be trusted

The AI mostly isn't writing SQL — it's filling in parameters for reviewed templates. Roughly 85% of checks are templated (just a config diff to review), 15% need a human to confirm the grain, and a small number of business-rule checks get full code review.

Before we trust a generated check, it has to pass four tests: it runs within budget; it passes on data we know is good; it correctly fails when we inject the exact bug it's meant to catch; and it gives the same result twice on frozen data.

A check that can never fail is worse than no check at all — it just creates false confidence. A wrong grain has the same problem: it usually still passes, because it's comparing nothing to nothing. That's why grain always needs a human to confirm it.

2.7 Handling expected differences

Manual exception lists go stale — an exception from March can quietly hide a real bug in July. We handle this four ways: exceptions expire automatically when the related code changes; we track which exact rows are failing, so new failures always need a fresh look; we check git history on both sides to tell an intentional change apart from a one-sided fix; and we ask the author directly in the pull request, while they still remember why.

Onboarding a project means writing about fifty lines of config — connections, where the code lives, column rules, ownership — not writing checks by hand. Checks are generated from known table types (raw, SCD1, SCD2, fact, mart, reference). The first project takes about a day plus two weeks of shadow monitoring before it goes live; every project after that takes a few hours.

3. Technology stack

Layer	Choice	Why
Language / config	Python 3.11+, YAML + Pydantic v2	Standard in data engineering; config is the product, so it's validated at load time
CLI	Typer	The runner is a CLI — cron, hooks, CI, and orchestrators can all call it directly
Warehouse access	Native driver behind a thin dialect layer	We generate SQL ourselves and need full control, not an ORM
SQL templating	Jinja2	Familiar from dbt; templates read like reviewable SQL
LLM	Claude via the Anthropic SDK, structured output via tool use	Usable output, not free text to parse; Bedrock/Vertex if residency requires
State store	A schema in the existing warehouse	No new infrastructure to provision, secure, or back up

Layer	Choice	Why
Reporting	Jira REST v3, Slack Web API with Block Kit	Lane A is unattended; Slack buttons let people acknowledge, flag false positives, or snooze
Triggers	Lane A: cron or a completion hook. Lane B: CLI and GitHub Actions on PR	The hook avoids the mid-load race; both Lane B paths are just the CLI
Dashboard	Streamlit for v0; FastAPI + React later	Deliberately thin — it reads state, it doesn't compute anything
Packaging / testing	Docker + pyproject.toml; pytest, SQL snapshot tests, scratch schema for mutation tests	One artifact everywhere; templates and mutation tests are the foundation of trust
Secrets	Env vars backed by the platform secret manager	Per-project isolation from day one

What we build vs. borrow: we build the comparison logic and the root-cause/routing logic, since that's where the value is. We skip cross-engine hash diffing entirely, and would borrow an existing tool if a second engine ever shows up. We're deliberately not building anomaly detection or drift detection — too noisy, and we don't have the history to make it reliable.

None of this depends on dbt, an orchestrator, a data catalog, or streaming infrastructure. If any of those exist, we use them as a signal, never a requirement — which is what keeps onboarding a second project cheap.

4. Scope

4.1 In scope

- Lane A: the SCD suite (11 checks), key/grain uniqueness, foreign key resolution, valid ranges and enums, cast-failure rate, conservation across layers, join-fanout guards, and business rules — scheduled, with full dev reporting.
- Lane B: a single SQL statement (full outer join / except) that returns the differing rows and columns, run on demand or via the PR gate, with a per-run cost budget.
- Run correctness: four-way column classification with default exclusions, reported every run; row scoping to completed batches; explicit numeric scale, UTC timestamps, NULL policy, and float tolerance.
- AI authoring: catalog discovery, table/column classification, manifest proposals, check generation with the four validation gates, all delivered as a git PR.
- AI reporting: deterministic evidence collection, closed-category classification, grouping related failures, ownership routing, Slack + Jira with feedback buttons.
- Operations: state store, sign-off tracking, staleness detection, coverage reporting, a shadow mode for Lane A, a health dashboard, and PII handling.

4.2 Deferred — to revisit later

- Cross-engine comparison and all hashing logic — only if a genuinely different engine becomes a source, and even then we'd borrow rather than build.
- Anomaly detection and learned thresholds — after we have two or three quarters of run history.
- Streaming sources — if a streaming ingest path appears.
- BI or reverse-ETL parity checks — once the warehouse side is trusted.
- A circuit breaker / dataset quarantine — once the state store itself is trusted.
- A cross-project dashboard or multi-tenant version — at project three, if this becomes a shared product.

4.3 Out of scope, on principle

This tool reports on data, it never changes it. No AI output goes live without review. AI never sits directly in the data path — a bad suggestion, not bad data, is the worst case. No writes to source systems. No blocking a deploy without buy-in first. And no requirement for dbt, an orchestrator, or a catalog to exist.

4.4 Minimum viable version

The SCD suite running on one real pipeline, on a schedule, with working Slack and Jira alerts; column rules with exclusions shown; row scoping in place; all four validation gates (especially the inject-a-bug test); and a coverage view.

5. Assumptions and considerations

Each one below notes what breaks if it turns out to be wrong. The bold ones need confirming before we start building.

One Assumption before anything is , the initial implementation is being planned with snowflake as the warehousing Platform for simplicity and familiarity.

#	Assumption	If wrong
A1	We get a read-only warehouse role, plus write access to one small state schema	Nothing works without the read role; without the write schema, state moves elsewhere and adds a security review to the critical path
A2	Both sources being compared live in the same engine	Cross-engine work re-enters scope — hashing, canonicalization, a borrowed diff engine. Substantially more work
A3	Pipeline code is in git and readable	We lose the ability to bind sign-offs to a commit, resolve ownership, or detect stale checks
A4	There's a reliable signal for when a batch has finished loading	Row scoping falls back to guessing by wall-clock time, and scheduled checks will

#	Assumption	If wrong
		sometimes read mid-load data and report false failures
A5	The warehouse exposes operational history, and we can call an LLM API	Root-cause analysis loses its best signals; authoring falls back to templates and guesswork
A6	No orchestrator is assumed to exist	No downside — if one does exist, it becomes a useful signal and a natural hook point
B1	Audit and ETL columns can be identified by naming convention or declaration	Checks pick up run-to-run noise on every row — the single most common cause of a useless first run
B2	Dimensions follow a recognisable SCD2 shape with a business key distinct from the surrogate key	The SCD suite can't be generated automatically. The most common misclassification, so it needs human confirmation
B3	Grain can be inferred from code and schema, with a human confirming it	A wrong grain produces a check that passes while comparing nothing
B4	Some differences between sources are deliberate and undocumented	Safe to assume — shadow mode exists to catch these before go-live
C1	Developers will actually act on Lane A reports, and each pipeline has a named owner	Without action, Lane A is just noise; without an owner, sign-offs get rubber-stamped just to keep the dashboard green
C2	We can add a non-blocking PR check	We lose the ability to capture intent up front — the single biggest hit to what this tool can do
C3	We get 30–60 minutes from a domain expert per project, plus a two-week shadow period	Onboarding either ships with the wrong grain, or goes live into a noisy baseline and wastes the team's first impression

Governance, security, and cost. PII is a real concern here, not a checkbox — the reporting agent sees the actual violating rows, which counts as data leaving the warehouse. We handle this with column-level tagging, hash-only comparison for sensitive columns, redaction before any LLM call, and a record of what left the perimeter. During authoring, we only ever look at metadata, never row values. We also fingerprint the tool's own permissions at startup, since a permission change can otherwise look exactly like data loss. On cost: Lane A is cheap by construction, Lane B has a hard per-run budget with an abort switch, and every run logs its own cost.

Limitations worth stating up front. Because Lane B runs on demand, a two-source mismatch is only caught when someone runs it — a deliberate tradeoff for cost, and the fix if it becomes a problem is a hook, not a cron job. Some source queries are non-deterministic by nature, and the tool needs to recognise and say so rather than flag a false failure. Floating-point comparisons use a tolerance, not exact equality. And coverage will never reach 100% — a green result means everything we checked, on the columns we compared, looks fine, which is why we always report exclusions and coverage alongside it.

6. Risks

Fourteen of the twenty risks in the full register are handled structurally, by a design decision rather than by relying on people to follow a process. The ones that aren't are worth raising directly.

#	Risk	How we handle it	By design?
R1	Audit columns included, so every row looks different	Default-exclude patterns; exclusions reported every run	Yes
R2	A check reads mid-load data	Scope to completed batches; prefer a post-run hook; log the predicate used	Yes
R3	Chronic red alerts that everyone learns to ignore	Dedup by failing rows; sign-offs bound to a commit; a mandatory shadow period	Yes
R4	A generated check that can never fail, or a wrong grain that always passes	Mandatory mutation testing; human confirmation of grain	Yes
R5	The LLM invents a root cause	Deterministic evidence gathered first; fixed categories; every output is reviewable	Yes
R6	One break files thirty tickets, or auto-assignment reads as blame	Group related failures at the earliest layer; assign to the team queue, just mention the likely author	Yes
R7	PII reaches the LLM, or a permission change looks like data loss	Tagging, hash-only comparison, redaction, an egress record; permissions fingerprinted at startup	Yes
R8	Nobody actually acts on Lane A reports	Confirm a named recipient before we build the reporting path	No

7. Success metrics

Type	Metric	Target
Leading	Mutation-test pass rate on active checks	100% — non-negotiable

Type	Metric	Target
	Runs reporting column exclusions; human-confirmed grain	100% each
	Coverage: objects checked vs. total in scope	Trending up, honestly reported
Trust	Precision: Lane A incidents confirmed real	Over 70% at launch, over 85% steady state
	Root-cause accuracy: predicted vs. confirmed	Over 60% at launch, improving
	Lane A alerts per engineer per week	Under five
Outcome	Time to detect and time to diagnose an issue	Hours, not days — and falling
	Monitoring cost per pipeline per month	Below the cost of one incident
	Onboarding cost, project two vs. project one	25% or less

Time to detection and issues caught pre-merge matter most, since they describe a real change in outcomes, not just activity. Precision and cost are what keep the tool trusted internally.

Deliberately not tracked: number of checks (500 that can't fail is worse than 40 that can), percentage of checks passing (rewards weak checks), number of tickets filed (rewards noise), and a single headline coverage percentage without the breakdown behind it.